გაკვეთილი 7 Mod 4 (4.2)

გაკვეთილი 5 — TypeScript-ში პირობითი ოპერატორები (if, else if, else)

1. წინა გაკვეთილის გახსენება

მეოთხე გაკვეთილში ვისწავლეთ შედარებისა და ლოგიკური ოპერატორები.

მაგალითად:

```
let age: number = 20;
let hasTicket: boolean = true;

let canEnter: boolean = age >= 18 && hasTicket;

console.log(canEnter);
```

შედეგი:

```
true
```

აქ `canEnter` ინახავს `boolean` მნიშვნელობას.

მაგრამ რა მოხდება, თუ გვინდა, რომ პირობის შესრულების შემთხვევაში ერთი მოქმედება შესრულდეს, ხოლო შეუსრულებლობის შემთხვევაში — მეორე?

ამისთვის გამოიყენება პირობითი ოპერატორები.

2. `if` — პირობითი ოპერატორი

`if` გამოიყენება მაშინ, როდესაც გვინდა, რომ გარკვეული კოდი შესრულდეს მხოლოდ კონკრეტული პირობის დაკმაყოფილების შემთხვევაში.

სინტაქსი:

```
if (condition) {
    // კოდი შესრულდება, თუ პირობა true-ა
}
```

მაგალითი 1 — ასაკის შემოწმება

```
let age: number = 20;

if (age >= 18) {
    console.log("You are an adult");
}
```

შედეგი:

```
You are an adult
```

პროგრამა ამოწმებს, არის თუ არა `age` 18-ზე მეტი ან ტოლი.

რადგან პირობა `true`-ა, სრულდება ფიგურულ ფრჩხილებში მოთავსებული კოდი.

მაგალითი 2 — ქულის შემოწმება

```
let score: number = 45;

if (score >= 50) {
    console.log("Passed");
}
```

ამ შემთხვევაში კონსოლში არაფერი გამოვა, რადგან პირობა `false`-ია.

TypeScript-ის თავისებურება

TypeScript-ში ცვლადის ტიპი წინასწარ არის განსაზღვრული:

```
let score: number = 45;
```

თუ შეცდომით ტექსტს მივანიჭებთ:

```
score = "60";
```

TypeScript შეცდომას გვაჩვენებს, რადგან `score` არის `number` ტიპის ცვლადი.

3. `if...else` — ორი შესაძლო შედეგი

`else` გამოიყენება მაშინ, როდესაც გვინდა, რომ პირობის შეუსრულებლობის შემთხვევაში სხვა კოდი შესრულდეს.

სინტაქსი:

```
if (condition) {
    // პირობა შესრულდა
} else {
    // პირობა არ შესრულდა
}
```

მაგალითი 1 — გამოცდის შედეგი

```
let score: number = 75;

if (score >= 50) {
    console.log("Passed");
} else {
    console.log("Failed");
}
```

შედეგი:

```
Passed
```

მაგალითი 2 — მომხმარებლის ავტორიზაცია

```
let isLoggedIn: boolean = false;

if (isLoggedIn) {
    console.log("Welcome back!");
} else {
    console.log("Please log in");
}
```

შედეგი:

```
Please log in
```

აქ `isLoggedIn` უკვე `boolean` ტიპისაა, ამიტომ დამატებით `=== true` შედარება აუცილებელი არ არის.

4. `else if` — რამდენიმე პირობის შემოწმება

`else if` გამოიყენება მაშინ, როდესაც რამდენიმე შესაძლო პირობა გვაქვს.

პროგრამა პირობებს ზემოდან ქვემოთ ამოწმებს და ასრულებს პირველივე დაკმაყოფილებული პირობის შესაბამის ბლოკს.

მაგალითი — შეფასების სისტემა

```
let score: number = 85;

if (score >= 90) {
    console.log("Excellent");
} else if (score >= 75) {
    console.log("Good");
} else if (score >= 50) {
    console.log("Passed");
} else {
    console.log("Failed");
}
```

შედეგი:

```
Good
```

რატომ?

1. `85 >= 90` — `false`.
2. `85 >= 75` — `true`.
3. პროგრამა გამოიტანს `"Good"`-ს.
4. დარჩენილი პირობები აღარ შემოწმდება.

პირობების თანმიმდევრობა

მნიშვნელოვანია, რომ პირობები სწორი თანმიმდევრობით დავწეროთ.

მაგალითად:

```
let score: number = 95;

if (score >= 50) {
    console.log("Passed");
} else if (score >= 90) {
    console.log("Excellent");
}
```

შედეგი იქნება:

```
Passed
```

მიუხედავად იმისა, რომ ქულა 95-ია.

ეს იმიტომ ხდება, რომ პირველი პირობა უკვე შესრულდა.

დასამახსოვრებელი: `else if` კონსტრუქციაში პირობების თანმიმდევრობას დიდი მნიშვნელობა აქვს.

5. ლოგიკური ოპერატორები პირობებში

წინა გაკვეთილში ვისწავლეთ `&&`, `||` და `!`.

ახლა ისინი გამოვიყენოთ `if` კონსტრუქციასთან ერთად.

მაგალითი 1 — AND (`&&`)

მომხმარებელს შეუძლია ღონისძიებაზე შესვლა, თუ სრულწლოვანია და ბილეთი აქვს.

```
let age: number = 20;
let hasTicket: boolean = true;

if (age >= 18 && hasTicket) {
    console.log("Access granted");
} else {
    console.log("Access denied");
}
```

შედეგი:

```
Access granted
```

მაგალითი 2 — OR (`||`)

მომხმარებელს შეუძლია სისტემაში შესვლა, თუ ადმინისტრატორია ან მოდერატორი.

```
let isAdmin: boolean = false;
let isModerator: boolean = true;

if (isAdmin || isModerator) {
    console.log("Access granted");
} else {
```

```
    console.log("Access denied");
}
```

შედეგი:

```
Access granted
```

მაგალითი 3 — NOT (`!`)

```
let isBanned: boolean = false;

if (!isBanned) {
    console.log("User is allowed");
} else {
    console.log("User is banned");
}
```

შედეგი:

```
User is allowed
```

6. ჩადგმული პირობები — Nested Conditions

ჩადგმული პირობა ნიშნავს ერთი `if` კონსტრუქციის მოთავსებას მეორე `if` კონსტრუქციის შიგნით.

მაგალითი — თამაშის დაწყება

```
let age: number = 19;
let hasAccount: boolean = true;

if (age >= 18) {
    if (hasAccount) {
        console.log("You can start the game");
    } else {
        console.log("Create an account first");
    }
} else {
    console.log("You must be at least 18");
}
```

შედეგი:

```
You can start the game
```

პროგრამის მუშაობა:

1. ჯერ მოწმდება ასაკი.
2. თუ მომხმარებელი სრულწლოვანია, მოწმდება ანგარიშის არსებობა.
3. ორივე პირობის შესრულების შემთხვევაში თამაში იწყება.

იგივე ლოგიკის დაწერა შეგვიძლია `&&` ოპერატორითაც, თუმცა ჩადგმული პირობები სასარგებლოა მაშინ, როდესაც სხვადასხვა შემთხვევაში განსხვავებული მოქმედებები გვჭირდება.

7. TypeScript-ის მთავარი თავისებურება — Type Narrowing

ახლა გადავიდეთ დღევანდელი გაკვეთილის ყველაზე მნიშვნელოვან TypeScript თემაზე.

Type Narrowing ნიშნავს ცვლადის შესაძლო ტიპების დაზუსტებას გარკვეული შემოწმების საფუძველზე.

წინა გაკვეთილებში ვისწავლეთ Union Type.

მაგალითად:

```
let userId: number | string;
```

ეს ნიშნავს, რომ `userId` შეიძლება იყოს `number` ან `string`.

მაგრამ როგორ გავიგოთ, კონკრეტულ მომენტში რომელი ტიპის მნიშვნელობა გვაქვს?

ამისთვის შეგვიძლია გამოვიყენოთ `typeof` და `if`.

მაგალითი 1 — ტიპის შემოწმება

```
let userId: number | string = "USER123";

if (typeof userId === "string") {
    console.log("User ID is text");
} else {
    console.log("User ID is number");
}
```

შედეგი:

```
User ID is text
```

აქ `typeof` ამოწმებს მნიშვნელობის რეალურ ტიპს.

როგორ მუშაობს Type Narrowing?

განვიხილოთ ფუნქცია, რომელიც იღებს `number | string` ტიპის მნიშვნელობას:

```
function showValue(value: number | string): void {
    if (typeof value === "string") {
        console.log(value.toUpperCase());
    } else {
        console.log(value * 2);
    }
}

showValue("hello");
showValue(10);
```

შედეგი:

```
HELLO
20
```

პირველ შემთხვევაში:

```
typeof value === "string"
```

TypeScript ხვდება, რომ ამ ბლოკში `value` არის `string`.

ამიტომ შეგვიძლია გამოვიყენოთ:

```
value.toUpperCase()
```

მეორე შემთხვევაში TypeScript უკვე იცის, რომ `value` არის `number`.

ამიტომ შეგვიძლია გამოვიყენოთ:

```
value * 2
```

რა მოხდება შემოწმების გარეშე?

```
function showValue(value: number | string): void {
    console.log(value.toUpperCase());
}
```

TypeScript შეცდომას გვაჩვენებს.

რატომ?

იმიტომ, რომ `value` შეიძლება იყოს `number`, ხოლო `number` ტიპს `toUpperCase()` მეთოდი არ აქვს.

Type Narrowing-ის მთავარი დანიშნულებაა სხვადასხვა შესაძლო ტიპის მქონე მნიშვნელობების უსაფრთხოდ გამოყენება.

8. `typeof` — Type Guard

TypeScript-ში ტიპის შემოწმების ერთ-ერთი გავრცელებული საშუალებაა `typeof`.

მას Type Guard-ის სახით ვიყენებთ.

ძირითადი მაგალითები:

```
typeof value === "string"
typeof value === "number"
typeof value === "boolean"
```

მაგალითი — სხვადასხვა ტიპის მონაცემების დამუშავება

```
function processData(data: number | string | boolean): void {
    if (typeof data === "number") {
        console.log(data + 10);
    } else if (typeof data === "string") {
        console.log(data.toUpperCase());
    } else {
        console.log(!data);
    }
}

processData(20);
processData("typescript");
processData(true);
```

შედეგი:

```
30
TYPESCRIPT
false
```

განვიხილოთ თითოეული შემთხვევა.

პირველი გამოძახება:

```
processData(20);
```

`data` არის `number`, ამიტომ ემატება 10.

მეორე გამოძახება:

```
processData("typescript");
```

`data` არის `string`, ამიტომ ტექსტი გარდაიქმნება დიდ ასოებად.

მესამე გამოძახება:

```
processData(true);
```

`data` არის `boolean`, ამიტომ `!data` აბრუნებს `false`-ს.

ეს მაგალითი აჩვენებს, როგორ შეგვიძლია ერთი ფუნქციით რამდენიმე ტიპის მონაცემის უსაფრთხოდ დამუშავება.

9. `null` მნიშვნელობის შემოწმება

TypeScript-ში ხშირად გვხვდება ისეთი ცვლადები, რომლებიც შეიძლება შეიცავდნენ `null` მნიშვნელობას.

მაგალითად:

```
let username: string | null = null;
```

ეს ნიშნავს, რომ `username` შეიძლება იყოს ტექსტი ან `null`.

განვიხილოთ ფუნქცია:

```
function greetUser(username: string | null): void {
    console.log(username.toUpperCase());
```

```
}
```

როდესაც TypeScript-ის `strictNullChecks` ჩართულია, ეს კოდი შეცდომას გვაჩვენებს.

რატომ?

იმიტომ, რომ `username` შეიძლება იყოს `null`, რომელსაც `toUpperCase()` მეთოდი არ აქვს.

სწორი ვარიანტი

```
function greetUser(username: string | null): void {
    if (username !== null) {
        console.log(username.toUpperCase());
    } else {
        console.log("Guest");
    }
}

greetUser("Giorgi");
greetUser(null);
```

შედეგი:

```
GIORGI
Guest
```

აქ `if` შემოწმების შემდეგ TypeScript-მა იცის, რომ პირველ ბლოკში `username` ნამდვილად `string` ტიპისაა.

ესეც Type Narrowing-ის მაგალითია.

10. TypeScript-ში `if` და Type Conversion

მესამე გაკვეთილში ვისწავლეთ მონაცემთა ტიპების გარდაქმნა.

ახლა ეს ცოდნა გამოვიყენოთ პირობით ოპერატორებთან ერთად.

მაგალითი — ასაკის შემოწმება

```
let ageText: string = "20";

let age: number = Number(ageText);

if (age >= 18) {
```

```
    console.log("Adult");
} else {
    console.log("Minor");
}
```

შედეგი:

```
Adult
```

მაგრამ რა მოხდება, თუ ტექსტში რიცხვი არ წერია?

```
let ageText: string = "Hello";

let age: number = Number(ageText);
```

შედეგი იქნება `NaN`.

ამიტომ მონაცემის შემოწმებაც საჭიროა.

```
let ageText: string = "Hello";

let age: number = Number(ageText);

if (Number.isNaN(age)) {
    console.log("Invalid age");
} else if (age >= 18) {
    console.log("Adult");
} else {
    console.log("Minor");
}
```

შედეგი:

```
Invalid age
```

ამ მაგალითში ერთად გამოვიყენეთ:

- Type Conversion;
- `if`;
- `else if`;
- `else`;
- TypeScript-ის ტიპები;

- `Number.isNaN()`.

11. პრაქტიკული სავარჯიშოები

სავარჯიშო 1 — ასაკის შემოწმება

მოცემულია:

```
let age: number = 16;
```

დავალება: დაწერეთ TypeScript პროგრამა, რომელიც შეამოწმებს მომხმარებლის ასაკს.

თუ ასაკი 18-ზე მეტი ან ტოლია, გამოიტანეთ `"Adult"`, წინააღმდეგ შემთხვევაში — `"Minor"`.

სწორი პასუხის ნახვა

სავარჯიშო 2 — შეფასების სისტემა

მოცემულია:

```
let score: number = 78;
```

დაწერეთ პროგრამა, რომელიც ქულის მიხედვით გამოიტანს:

- 90–100 — `"Excellent"`;
- 75–89 — `"Good"`;
- 50–74 — `"Passed"`;
- 0–49 — `"Failed"`;
- სხვა მნიშვნელობის შემთხვევაში — `"Invalid score"`.

გამოიყენეთ `if`, `else if` და `else`.

სწორი პასუხის ნახვა

სავარჯიშო 3 — Type Narrowing

მოცემულია ფუნქცია:

```
function checkValue(value: number | string): void {

}
```

დავალება: შეავსეთ ფუნქცია ისე, რომ:

1. თუ `value` არის `number`, გამოიტანოს მისი გაორმაგებული მნიშვნელობა.
2. თუ `value` არის `string`, გამოიტანოს ტექსტი დიდი ასოებით.
3. გამოიყენოთ `typeof`.

ფუნქცია გამოიძახეთ შემდეგი მნიშვნელობებით:

```
checkValue(15);
checkValue("hello");
```

მოსალოდნელი შედეგი:

```
30
HELLO
```

სწორი პასუხის ნახვა

სავარჯიშო 4 — შეცდომის პოვნა

მოცემულია:

```
function showUsername(username: string | null): void {
    console.log(username.toUpperCase());
}
```

დავალება:

1. იპოვეთ TypeScript-ის შეცდომა.
2. ახსენით, რატომ შეიძლება წარმოიშვას პრობლემა.
3. გაასწორეთ ფუნქცია ისე, რომ `null` მნიშვნელობის შემთხვევაში გამოიტანოს `"Guest"`.

სწორი პასუხის ნახვა

სავარჯიშო 5 — მომხმარებლის წვდომის სისტემა

შექმენით TypeScript პროგრამა, რომელიც შეამოწმებს მომხმარებლის უფლებებს.

მოცემულია:

```
let age: number = 20;
let isAdmin: boolean = false;
```

```
let hasSubscription: boolean = true;
let isBanned: boolean = false;
```

პროგრამამ უნდა შეასრულოს:

1. თუ მომხმარებელი დაბლოკილია, გამოიტანოს `"Access denied"`.
2. თუ მომხმარებელი ადმინისტრატორია, გამოიტანოს `"Admin access"`.
3. თუ მომხმარებელი სრულწლოვანია და აქვს გამოწერა, გამოიტანოს `"Access granted"`.
4. სხვა შემთხვევაში გამოიტანოს `"Subscription required"`.

გამოიყენეთ `if`, `else if`, `else`, `&&` და `!` ან შესაბამისი ლოგიკური შემოწმება.

სწორი პასუხის ნახვა

12. დამატებითი პრაქტიკული დავალება — ონლაინ მაღაზიის შეკვეთა

შექმენით TypeScript პროგრამა, რომელიც ამოწმებს, შეუძლია თუ არა მომხმარებელს პროდუქტის შეძენა.

მოცემულია:

```
let priceText: string = "150";
let quantity: number = 2;
let balance: number = 400;
let isAvailable: boolean = true;
let discountCode: string | null = "SALE";
```

პროგრამამ უნდა:

1. `priceText` გარდაქმნას `number` ტიპად.
2. გამოთვალოს პროდუქციის ჯამური ღირებულება.
3. თუ `discountCode` არის `"SALE"`, ჯამურ ღირებულებას გამოაკლოს 50.
4. თუ პროდუქტი არ არის მარაგში, გამოიტანოს `"Out of stock"`.
5. თუ მომხმარებელს საკმარისი თანხა არ აქვს, გამოიტანოს `"Not enough money"`.
6. წინააღმდეგ შემთხვევაში გამოიტანოს `"Purchase successful"`.
7. წარმატებული შეძენის შემთხვევაში გამოიტანოს საბოლოო ფასიც.